

CMSC 471 - Programming Assignment 1

Mobolaji Ogunbiyi

Question 1 - Heuristics

Let $d(n,g)$ be the number of letter positions where the current word n differs from goal g . All three heuristics return 0 at the goal.

Steps: $h(n) = d(n,g)$. Each mismatched position must be changed at least once, and each change costs 1. Extra changes can only add cost.

Scrabble: $h(n)$ is the sum of the Scrabble values of $g[i]$ over positions i where $n[i]$ differs from $g[i]$. To finish, each such position must receive its goal letter at least once. That insertion has the listed cost; other insertions only add cost.

Frequency: If $n = g$, $h(n) = 0$. Otherwise $h(n) = d(n,g) + \text{rarity}(g)$. At least d changes remain, each costing at least 1 because rarity is nonnegative. The final change enters g and pays $\text{rarity}(g)$. Thus both parts of the estimate are unavoidable.

Question 2 - UCS and A*

Word pair	Cost	UCS cost	UCS expanded	A* cost	A* expanded	Reduction
WHY to ASK	Steps	5	773	5	16	97.93%
WHY to ASK	Scrabble	9	765	9	13	98.30%
WHY to ASK	Frequency	38.8554	808	38.8554	634	21.53%
BEAR to DUCK	Steps	4	691	4	6	99.13%
BEAR to DUCK	Scrabble	11	2981	11	6	99.80%
BEAR to DUCK	Frequency	38.2402	1084	38.2402	302	72.14%

Reduction = $100 \times (\text{UCS expanded} - \text{A* expanded}) / \text{UCS expanded}$. Counts are the values printed by the supplied search functions with `display=True`. Each run used a new DC instance.

UCS and A* returned the same optimal cost in all six runs. For both word pairs, Scrabble gave the largest reduction and frequency gave the smallest. The Scrabble bound prices every goal letter that still needs to be inserted, so it rules out many expensive routes. The frequency bound counts required changes and the goal entry rarity, but says little about the rarity of intermediate words. Those unpriced intermediate costs leave more nodes looking competitive.

Question 3 - Reversibility and cost

Steps: Yes. Reversing any path uses the same number of actions. The minimum number of actions is therefore equal in both directions.

Scrabble: No. DOG to COG costs 3 because C is inserted. COG to DOG costs 2 because D is inserted. Both are legal dictionary words, and a direct one-step path is already cheapest in each direction.

Frequency: No. DOG to COG costs $1 + \text{rarity}(\text{COG}) = 12.393166$. COG to DOG costs $1 + \text{rarity}(\text{DOG}) = 6.819885$. A direct edge reaches the goal and no path can cost less than the cost of entering that goal. The moves are reversible, but their costs depend on which word is entered.

Question 4 - Generative AI heuristic audit

Model: GPT-6. **Prompt:**

Can you suggest a rarity-aware heuristic for DOGCAT? The words are lowercase and have three or four letters. Every legal word must be in the supplied dictionary, and each move changes exactly one letter. For the frequency cost, entering a word w costs $1 + \text{rarity}(w)$, where rarity is never negative.

The basic heuristic $h_0(n)$ counts the letters that differ from the goal. I want a heuristic that is at least as large as h_0 for every state and larger for at least one state, while never overestimating the cheapest remaining path. Please give a precise formula and explain why it is safe.

GPT-6 suggested adding the goal word's rarity to the number of letters that still differ: $h_{AI}(n) = d(n, g) + \text{rarity}(g)$ when n is not the goal, and $h_{AI}(g) = 0$.

I think this is safe. If d letters are wrong, I have to make at least d moves, and each move costs at least 1. I also have to enter the goal on the last move, which adds its rarity. So the estimate cannot be higher than the cheapest real path. It is 0 at the goal. Because rarity cannot be negative, it is at least as large as h_0 . ASK has rarity 4.2326436 and DUCK has rarity 7.2297187, so it is strictly larger than h_0 before reaching either goal. If a goal had rarity 0, this particular idea would only tie h_0 . I did not need to repair the heuristic for these two problems.

Frequency problem	h_0 cost	h_0 expanded	h_{AI} cost	h_{AI} expanded
WHY to ASK	38.8554	755	38.8554	634
BEAR to DUCK	38.2402	720	38.2402	302

The solution cost stayed the same for both pairs. With the added goal rarity, A^* expanded 634 nodes instead of 755 for WHY to ASK and 302 instead of 720 for BEAR to DUCK. It was more useful than h_0 on these two problems.

Sources: Supplied PA1 assignment, dictionary, and AIMA search files. Expansion counts came from running the submitted code with the supplied search functions.